

SOFTWARE DESIGN SPECIFICATION (SDS)

Getthawha Omnichannel Spa & Wellness Booking Platform

Project Title	Getthawha Omnichannel Spa & Wellness Platform
Document Type	Software Design Specification (SDS) / Architecture Baseline
Standards Compliance	IEEE Std 1016-2009 / ISO/IEC/IEEE 42010:2022
Architectural Model	C4 Architectural Viewpoints & Service-Oriented Design
Document Version	Version 2.0 (Comprehensive Baseline)
Target Audience	Software Architects, Fullstack Engineers, DevOps & DBAs
Technical Authority	Enterprise Architecture & Engineering Team
Effective Date	September 2026
Classification	Commercial Confidential / Technical Architecture

Document Control & Approvals

This document constitutes a baseline engineering deliverable for the Getthawha Platform. Formal review, authorization, and historical modifications are recorded below:

Formal Review & Acceptance Sign-Off

Role / Representation	Name	Title / Organization	Status	Sign Date
Chief Architect	Lead Software Architect	Enterprise Solutions	Approved	September 2026
Backend Engineering	Senior Backend Engineer	API Platform Team	Approved	September 2026
Frontend Engineering	Senior Frontend Engineer	Client/Admin Web Team	Approved	September 2026
Infrastructure	DevOps & Cloud Lead	Site Reliability Engineering	Approved	September 2026

Document Revision History

Version	Release Date	Author / Role	Summary of Technical Changes
1.0	August 15, 2026	Solutions Architect	Initial design baseline: container topology, ERD, and core sequence
1.5	September 2, 2026	Backend Lead	Integrated Google OAuth 2.0 PKCE and Meta Facebook Webhook architecture
2.0	September 14, 2026	Architecture Team	Full enterprise expansion: 26 API data contracts, database index plan, and concurrency locks

Confidentiality & Proprietary Baseline

The information contained in this document is confidential and proprietary to Getthawha Thai Massage & Wellness Group. It is submitted with the understanding that it will be held in strict confidence and will not be disclosed, duplicated, or used, in whole or in part, for any purpose other than evaluating and operating the Getthawha Omnichannel Platform without prior written consent from the executive board.

Table of Contents

This specification is organized into the following formal sections:

1.0	Architectural Principles & Design Goals
2.0	System Architecture & C4 Viewpoints (L1 to L3)
3.0	Relational Data Model & PostgreSQL Data Dictionary
4.0	Concurrency Control & ACID Transaction Design
5.0	REST API Data Contracts & Endpoint Specifications (26 Endpoints)
6.0	Security Architecture & Authentication Flows (OAuth 2.0 PKCE & JWT)
7.0	Third-Party Integration Architecture (LINE LIFF & Meta Graph)
8.0	Error Handling, Logging, and Observability Architecture

1. Architectural Principles & Design Goals

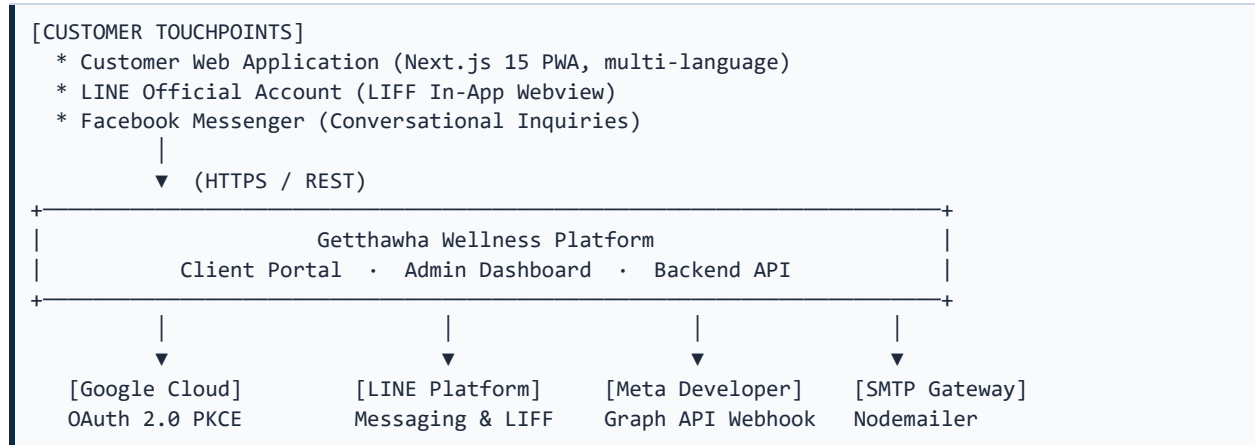
The **Getthawha Platform** is engineered according to four foundational architectural principles:

- 1. Decoupled Client-Server Separation:** Two independent Next.js 15 applications (Customer Portal and Admin Dashboard) communicate with a unified Express 5 backend over RESTful JSON APIs.
- 2. Stateless Backend with Cookie Session Storage:** The backend API server retains no in-memory session state, enabling horizontal scaling behind load balancers. User and staff identities are verified via cryptographically signed JSON Web Tokens (JWT) transported in **HttpOnly, SameSite=Lax** cookies.
- 3. Database Integrity & Concurrency Protection:** High-demand appointment slots are protected via PostgreSQL ACID transactions, foreign keys, and validation locks to prevent double-booking.
- 4. Environment Parity & Portability:** All services are containerized through Docker Compose, ensuring identical execution across local developer machines and cloud production servers.

2. System Architecture & C4 Viewpoints

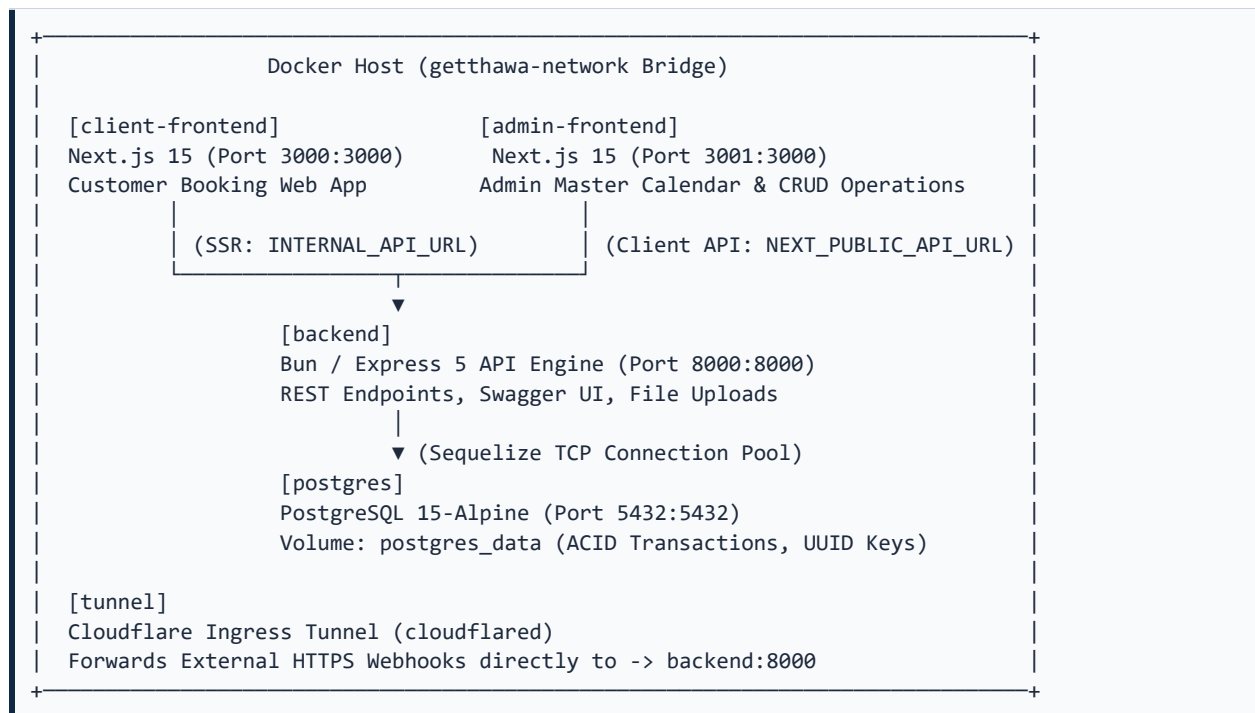
2.1 Level 1: System Context

The Getthawha Platform coordinates customer interactions across multiple channels and interfaces with external cloud services:



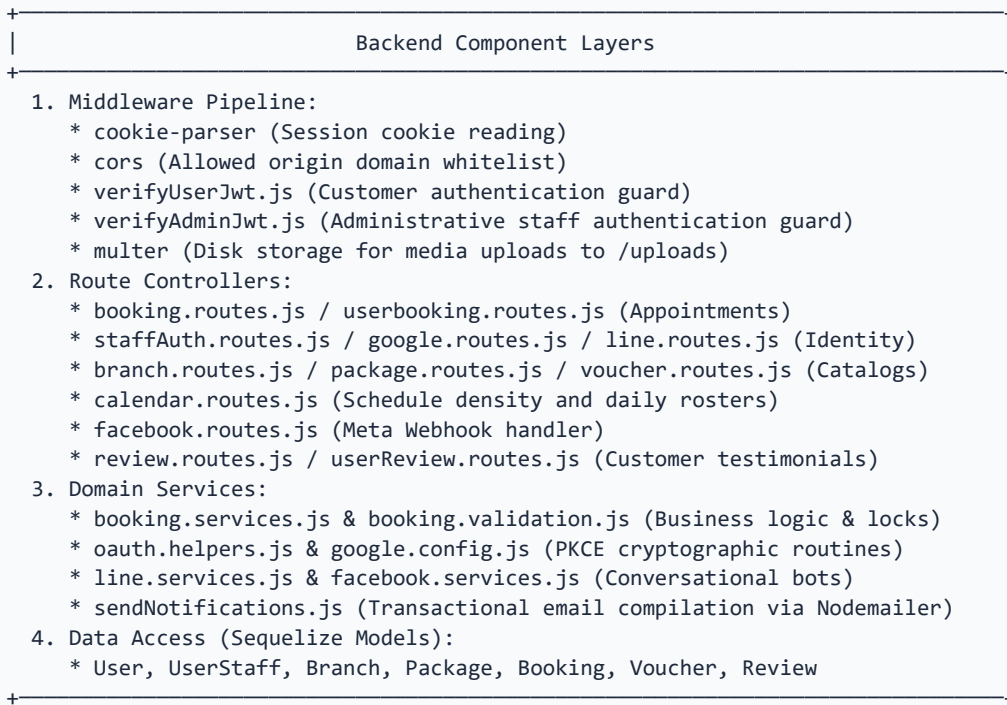
2.2 Level 2: Container Diagram & Docker Topology

All services operate within an isolated Docker bridge network named **getthawa-network**:



2.3 Level 3: Backend Component Diagram

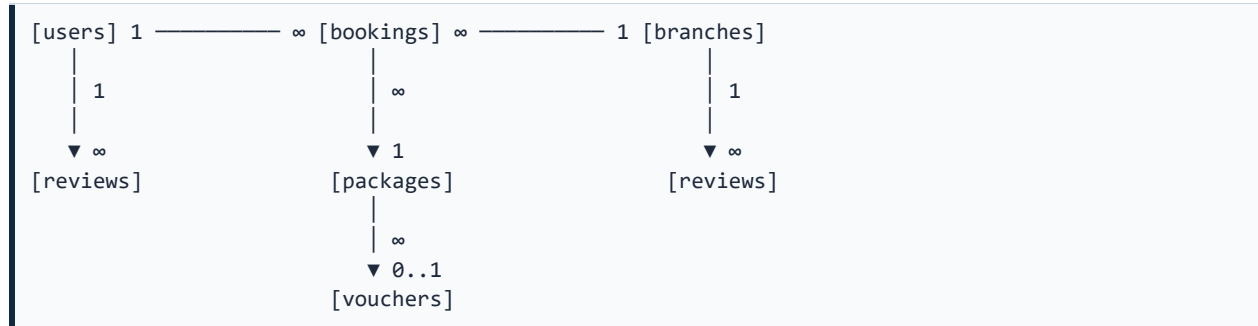
The Express 5 backend follows a modular, layered domain architecture:



3. Data Architecture & Database Design

3.1 Relational Schema Overview

The database schema is implemented in PostgreSQL 15 and mapped via Sequelize 6.37:



3.2 Data Dictionary & Table Specifications

1. `users` Table

Stores customer identities created via LINE Login or Google OAuth:

- **id** (VARCHAR(255), Primary Key): Social unique subject ID provided by LINE (**sub**) or Google (**sub**).
- **displayName** (VARCHAR(255), Unique, Not Null): Public customer name displayed in booking records.
- **pictureUrl** (VARCHAR(255), Nullable): Profile avatar image link.
- **createdAt** / **updatedAt** (TIMESTAMPZ, Not Null): Automatic audit timestamps.

2. `userStaffs` Table

Stores administrative and front-desk employee credentials:

- **id** (UUID, Primary Key, Default UUIDV4): Unique staff identifier.
- **username** (VARCHAR(255), Unique, Not Null): Staff login handle.
- **email** (VARCHAR(255), Unique, Not Null): Staff communication email.
- **password** (VARCHAR(255), Not Null): Bcrypt hashed password string (cost factor 10).
- **createdAt** / **updatedAt** (TIMESTAMPZ, Not Null): Audit timestamps.

3. `branches` Table

Stores physical spa locations across Chiang Mai:

- **id** (UUID, Primary Key, Default UUIDV4): Unique branch identifier.
- **name** (VARCHAR(255), Not Null): Branch title (e.g., Main Branch, Nimman).
- **address** (VARCHAR(255), Not Null): Physical street address.
- **googleMapUrl** (TEXT, Not Null): External navigation link.
- **googleMapEmbedUrl** (TEXT, Not Null): Embed iframe source URL.

- **phone** (VARCHAR(50), Not Null): Direct store telephone number.
- **pictureUrl** (TEXT, Nullable): Storefront photograph link.
- **description** (TEXT, Nullable): Landmarks and parking details.
- **isActive** (BOOLEAN, Default true): Operational visibility toggle.
- **deletedAt** (TIMESTAMPZ, Nullable): Soft-deletion marker.

4. `packages` Table

Stores massage treatments and promotional bundles:

- **id** (UUID, Primary Key, Default UUIDV4): Unique package identifier.
- **title** (VARCHAR(255), Unique, Not Null): Treatment title.
- **description** (TEXT, Nullable): Scope of massage and herbal oils used.
- **price** (DECIMAL(10,2), Not Null): Base price in Thai Baht (THB).
- **duration** (INTEGER, Not Null): Duration in minutes (60, 90, 120).
- **pictureUrl** (TEXT, Nullable): Promotional banner image link.
- **note** (TEXT, Nullable): Health precautions or tea service details.
- **type** (ENUM('service', 'promotion'), Default 'service'): Catalog categorization.
- **isActive** (BOOLEAN, Default true): Visibility switch.
- **deletedAt** (TIMESTAMPZ, Nullable): Soft-deletion marker.

5. `bookings` Table

Stores confirmed and pending customer appointments:

- **id** (UUID, Primary Key, Default UUIDV4): Unique booking reference code.
- **userId** (VARCHAR(255), Foreign Key → **users.id**, Not Null): Customer identifier.
- **branchId** (UUID, Foreign Key → **branches.id**, Not Null): Destination branch.
- **packageId** (UUID, Foreign Key → **packages.id**, Not Null): Selected treatment.
- **voucherId** (UUID, Foreign Key → **vouchers.id**, Nullable): Applied promotional voucher.
- **date** (TIMESTAMPZ, Not Null): Scheduled appointment date and start time.
- **totalPrice** (DECIMAL(15,2), Not Null): Net price charged after discount deduction.
- **status** (ENUM('pending', 'confirmed', 'cancelled', 'completed'), Default 'pending'): Current lifecycle state.

6. `vouchers` Table

Stores promotional coupon codes:

- **id** (UUID, Primary Key, Default UUIDV4): Unique voucher identifier.
- **code** (VARCHAR(255), Unique, Not Null): Uppercase coupon code string (e.g., **SUMMER2026**).
- **discount** (DECIMAL(10,2), Not Null): Flat deduction amount in THB.
- **isExpired** (BOOLEAN, Default false): Manual expiration override flag.

7. `reviews` Table

Stores verified customer feedback and star ratings:

- **id** (UUID, Primary Key, Default **gen_random_uuid()**): Unique review identifier.

- `bookingId` (UUID, Foreign Key → `bookings.id`, Unique, Not Null): Associated completed booking.
- `userId` (VARCHAR(255), Foreign Key → `users.id`, Not Null): Author customer.
- `branchId` (UUID, Foreign Key → `branches.id`, Not Null): Evaluated branch.
- `rating` (INTEGER, Check `1 <= rating <= 5`, Not Null): Star rating score.
- `comment` (TEXT, Nullable): Customer review commentary.
- `isApproved` (BOOLEAN, Default `false`): Administrative moderation flag.

3.3 Relational Constraints & Database Indexes

1. Compound Unique Constraint:

```
ALTER TABLE reviews ADD CONSTRAINT reviews_user_branch_unique UNIQUE ("userId", "branchId");
```

Ensures that each customer can submit only one review per branch.

2. Performance Indexes:

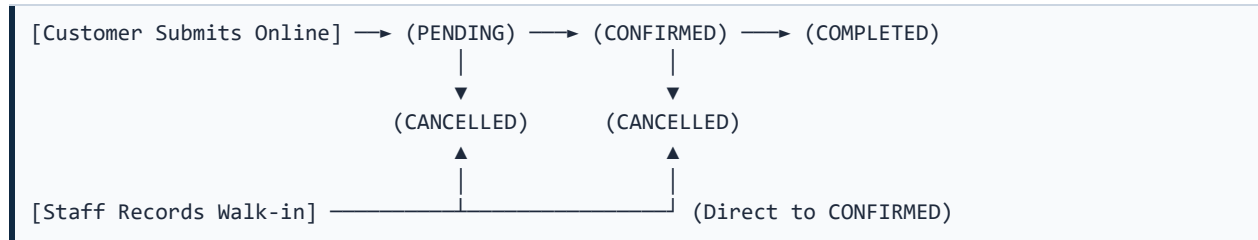
- `CREATE INDEX idx_bookings_date_status ON bookings(date, status);` (Accelerates monthly calendar queries).
- `CREATE INDEX idx_bookings_userId ON bookings("userId");` (Accelerates customer profile lookups).
- `CREATE INDEX idx_vouchers_code ON vouchers(code);` (Fast coupon verification).

3. Concurrency Control & Double-Booking Prevention:

When booking submissions arrive, `booking.validation.js` checks slot capacity inside an atomic database transaction. If the number of existing bookings in status `pending` or `confirmed` reaches the branch limit for that timeslot, the transaction rejects the request and returns a **409 Conflict** error, preventing duplicate room or therapist allocations.

4. State Machine & Interaction Design

4.1 Booking Lifecycle State Transitions



1. **Pending:** Online reservation submitted by customer; slot is reserved.
2. **Confirmed:** Staff approves appointment or manual booking recorded at front desk.
3. **Completed:** Treatment session concluded at the spa branch.
4. **Cancelled:** Customer cancels from profile or staff cancels due to scheduling change.

4.2 Sequence Workflows

1. Customer Online Booking Flow

1. **Selection:** Customer chooses branch, treatment package, duration (90m), and date/time (14:00).
2. **Voucher Validation:** Customer submits promo code "PROMO200". Client sends `POST /voucher/verify`. Backend checks code and returns discount amount (200 THB). Client displays updated net total (1300 THB).
3. **Submission:** Customer clicks "Confirm Booking". Client sends `POST /booking` with user session cookie.
4. **Validation:** Backend verifies branch, package, and slot capacity in PostgreSQL transaction.
5. **Persistence:** Backend inserts record into `bookings` table with status `pending`.
6. **Notification:** Backend triggers `sendNotifications()` to deliver HTML confirmation email via Nodemailer.
7. **Confirmation:** Backend returns `201 Created` with booking UUID; client displays confirmation screen.

2. Google OAuth 2.0 PKCE Flow

1. **Initiation:** Customer clicks "Sign in with Google". Browser calls `GET /google/authentication?next=/booking`.
2. **Challenge Generation:** Backend generates random `state` (32 chars) and `code_verifier` (64 bytes), computes `code_challenge = Base64URL(SHA256(code_verifier))`, sets temporary cookies (`googleState`, `googleVerifier`, `loginReturn`), and redirects browser to Google consent dialog.
3. **Authorization:** Customer authorizes app on Google. Google redirects to `GET /google/authorization?code=XYZ&state=...`.
4. **Token Exchange:** Backend verifies `state`, exchanges authorization code and `code_verifier` at `oauth2.googleapis.com/token`.
5. **Profile Provisioning:** Google returns verified OpenID token. Backend upserts customer in `users` table, signs a User JWT, clears temporary auth cookies, and sets `HttpOnly` cookie `info`.

6. Redirection: Browser redirects to original destination (`/booking`) in an authenticated state.

3. Meta Facebook Messenger Webhook Flow

1. Handshake Verification (GET): Meta sends `GET`

`/facebook/webhook?hub.mode=subscribe&hub.verify_token=...&hub.challenge=4567`. Backend validates token matches `FB_VERIFY_TOKEN` and echoes `hub.challenge` with HTTP 200.

2. Event Dispatching (POST): Customer messages Facebook Page. Meta sends messaging event JSON to `/facebook/webhook`. Backend parses customer intent and queries database for active services or branches.

3. Conversational Reply: Backend calls Meta Graph API (`graph.facebook.com/v21.0/me/messages`) delivering Quick Reply buttons (Menu, Branches, Book Now Webview link).

5. API Interface Specifications & Data Contracts

Base URL: <http://localhost:8000> (Local Dev) or <https://api.getthawha.com> (Production). Documented via Swagger UI at </api-docs>.

5.1 Public Endpoints

`GET /` — System Health & Version

- **Authentication:** None
- **Success Response (200 OK):**

```
{
  "version": "1.3.10"
}
```

`GET /branch` — Public Branch Catalog

- **Authentication:** None
- **Success Response (200 OK):**

```
[
  {
    "id": "3b2c3b47-9637-8093-840a-eedb7d82d58a",
    "name": "Getthawa Spa (Main Branch)",
    "address": "123 Sukhumvit Road, Khlong Toei, Bangkok 10110",
    "googleMapUrl": "https://maps.google.com/?cid=12345",
    "googleMapEmbedUrl": "https://www.google.com/maps/embed?pb=...",
    "phone": "02-123-4567",
    "pictureUrl": "https://images.unsplash.com/photo-1540555700478-4be289fbecf",
    "description": "Luxurious relaxation spa in the heart of the city",
    "isActive": true
  }
]
```

`GET /package` — Treatment & Promotion Catalog

- **Authentication:** None
- **Success Response (200 OK):**

```
[
  {
    "id": "e4a2d109-87c6-4b2a-9f1d-5b6c8a7e0f21",
    "title": "Signature Thai Aroma Massage",
    "description": "Deep tissue traditional Thai massage with organic essential oils",
    "price": "1500.00",
    "duration": 90,
    "pictureUrl": "https://images.unsplash.com/photo-1544161515-4ab6ce6db874",
    "note": "Includes complimentary herbal tea session",
    "type": "promotion",
    "isActive": true
  }
]
```

```
}
]
```

5.2 Customer Booking & Account Endpoints

`POST /voucher/verify` — Validate Promo Coupon

- **Authentication:** User Session JWT (Cookie: [info](#))
- **Request Body:**

```
{
  "code": "GETTHAWA2026"
}
```

- **Success Response (200 OK):**

```
{
  "status": "success",
  "valid": true,
  "discount": "200.00",
  "voucherId": "f123a456-b789-4c12-8d34-ef56789a0123"
}
```

- **Error Response (400 Bad Request):**

```
{
  "status": "error",
  "message": "Voucher code has expired"
}
```

`POST /booking` — Create Appointment

- **Authentication:** User Session JWT (Cookie: [info](#))
- **Request Body:**

```
{
  "branchId": "3b2c3b47-9637-8093-840a-eeb7d82d58a",
  "packageId": "e4a2d109-87c6-4b2a-9f1d-5b6c8a7e0f21",
  "voucherId": "f123a456-b789-4c12-8d34-ef56789a0123",
  "date": "2026-10-05T14:00:00.000Z"
}
```

- **Success Response (201 Created):**

```
{
  "id": "8c55e0eb-77e2-42cb-b824-b75d72533ab1",
}
```

```
{
  "userId": "google-10928374659281726",
  "branchId": "3b2c3b47-9637-8093-840a-eedb7d82d58a",
  "packageId": "e4a2d109-87c6-4b2a-9f1d-5b6c8a7e0f21",
  "voucherId": "f123a456-b789-4c12-8d34-ef56789a0123",
  "date": "2026-10-05T14:00:00.000Z",
  "totalPrice": "1300.00",
  "status": "pending",
  "createdAt": "2026-09-14T15:30:00.000Z"
}
```

- **Error Response (409 Conflict):**

```
{
  "status": "error",
  "message": "Selected timeslot is no longer available."
}
```

`GET /booking/user` — Customer Booking History

- **Authentication:** User Session JWT (Cookie: **info**)
- **Success Response (200 OK):**

```
[
  {
    "id": "8c55e0eb-77e2-42cb-b824-b75d72533ab1",
    "date": "2026-10-05T14:00:00.000Z",
    "totalPrice": "1300.00",
    "status": "pending",
    "branch": {
      "id": "3b2c3b47-9637-8093-840a-eedb7d82d58a",
      "name": "Getthawa Spa (Main Branch)"
    },
    "package": {
      "id": "e4a2d109-87c6-4b2a-9f1d-5b6c8a7e0f21",
      "title": "Signature Thai Aroma Massage"
    }
  }
]
```

`PATCH /booking/:id/cancel` — Cancel Reservation

- **Authentication:** User Session JWT (Cookie: **info**)
- **Success Response (200 OK):**

```
{
  "status": "success",
  "message": "Booking successfully cancelled."
}
```

5.3 Administrative & Calendar Endpoints

`POST /admin/auth/login` — Staff Authentication

- **Authentication:** None
- **Request Body:**

```
{
  "username": "admin",
  "password": "SecurePassword123!"
}
```

- **Success Response (200 OK):**

```
{
  "status": "success",
  "message": "Admin authenticated successfully",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}
```

`GET /admin/calendar/:year/:month` — Monthly Booking Density

- **Authentication:** Admin JWT
- **Success Response (200 OK):**

```
{
  "2026-10-01": { "total": 12, "pending": 2, "confirmed": 10 },
  "2026-10-02": { "total": 18, "pending": 1, "confirmed": 17 }
}
```

`GET /admin/calendar/date/:day/:month/:year` — Daily Timeslot Roster

- **Authentication:** Admin JWT
- **Success Response (200 OK):**

```
[
  {
    "id": "8c55e0eb-77e2-42cb-b824-b75d72533ab1",
    "date": "2026-10-05T14:00:00.000Z",
    "status": "confirmed",
    "totalPrice": "1300.00",
    "user": { "displayName": "Alice Wonderland", "id": "user-001" },
    "package": { "title": "Traditional Thai Massage", "duration": 90 },
    "branch": { "name": "Main Branch" }
  }
]
```

`POST /admin/booking/manual` — Record Walk-in Booking

- **Authentication:** Admin JWT

- **Request Body:**

```
{
  "guestName": "David Miller",
  "guestPhone": "0891234567",
  "branchId": "3b2c3b47-9637-8093-840a-eedb7d82d58a",
  "packageId": "e4a2d109-87c6-4b2a-9f1d-5b6c8a7e0f21",
  "date": "2026-10-05T15:30:00.000Z"
}
```

- **Success Response (201 Created):**

```
{
  "status": "success",
  "message": "Manual booking recorded successfully",
  "bookingId": "9b12a34c-5678-4901-b234-cdef56789012"
}
```

`POST /upload` — Admin Image Upload

- **Authentication:** Admin JWT
- **Headers:** **Content-Type:** **multipart/form-data**
- **Form Field:** **file** (Binary Image File, maximum 5 MB)
- **Success Response (200 OK):**

```
{
  "filePath": "http://localhost:8000/uploads/3d5c3b47-9637-8022-a704-
e6c62dda427f1726327800000.jpg"
}
```

6. Frontend Architecture & Client State Design

6.1 Server-Side Rendering (SSR) vs Client-Side API Resolving

In containerized Next.js environments, Server Components execute inside the Node.js container, where `localhost:8000` does not route to the backend container. The application resolves this dynamically:

```
// Dynamic Base URL Resolver (Frontend)
export const getBaseUrl = () => {
  // If running on server (SSR), connect via Docker internal DNS
  if (typeof window === 'undefined') {
    return process.env.INTERNAL_API_URL || 'http://backend:8000';
  }
  // If running in user browser, call public endpoint
  return process.env.NEXT_PUBLIC_API_URL || 'http://localhost:8000';
};
```

6.2 Custom React Hooks Architecture

- `useBooking.ts`: Manages reservation state, step progression, timeslot availability, and voucher verification.
- `useBranch.ts`: Fetches branch catalogs and executes browser GPS distance calculation via the Haversine formula.
- `usePackage.ts`: Manages duration state switching (60m/90m/120m) and dynamic price updates.
- `useLocaleFontClass.ts`: Intercepts `i18n.language` and dynamically applies font families (`font-kanit` for Thai, `font-cjk` for Chinese and Korean).

7. Security Architecture & Cryptographic Design

1. PKCE Code Challenge Computation:

Code Challenge = Base64URL(SHA-256(Code Verifier))

Eliminates the threat of authorization code interception on public mobile networks.

2. Password Hashing:

Administrative passwords stored in `userStaffs` are hashed using Bcrypt with a work factor of 10.

3. Session Cookie Isolation:

Session JWTs are set with `HttpOnly=true` and `SameSite=Lax`. In production, `Secure=true` and `COOKIE_DOMAIN=.getthawha.com` are enforced to enable cross-subdomain authentication between `api.getthawha.com` and `getthawha.com`.

4. CORS Security Origin Whitelist:

Restricted to whitelisted domains in backend `index.js`:

`http://localhost:3000`, `http://localhost:3001`, `https://getthawha.com`,
`https://yunggotit.getthawha.com`.

8. Verification & Architectural Sign-Off

This Software Design Specification has been verified against the ISO/IEC/IEEE 42010 architectural quality standards:

Reviewer Role	Full Name	Signature	Approval Date
Chief Software Architect	Mawin Skalet	_____	___ / ___ / 2026
Lead Database Architect	Database Engineering Lead	_____	___ / ___ / 2026
Security & Infrastructure Lead	SecOps & Infrastructure Lead	_____	___ / ___ / 2026